



Dynamic dispatch and polymorphism

▼ Type	 Lecture
📅 Date	@January 8, 2022
☰ Lecture #	3
🔗 Lecture URL	https://youtu.be/zN-6WryuloU
🔗 Notion URL	https://21f1003586.notion.site/Dynamic-dispatch-and-polymorphism-Qd56d1e9d22a468d9f2fd3744afd0b25
# Week #	3

Subclasses and inheritance

- A subclass extends a parent class
- Subclass inherits instance variables and methods from the parent class
- Subclasses cannot see private components of the parent class
- Subclass can add more instance variables and methods
- Can also override methods

```
public class Employee {
    private String name;
    private double salary;

    public boolean setName(String s) { ... }
    public boolean setSalary(double x) { ... }
    public String getName() { ... }
    public double getSalary() { ... }

    public double bonus(float percent) {
        return (percent/100.0) * salary;
    }
}

public class Manager {
    private String secretary;
    public boolean setSecretary(String s) { ... }
    public String getSecretary() { ... }
}
```

Dynamic dispatch

- `Manager` can redefine `bonus()`

```
double bonus(float percent) {
    return 1.5 * super.bonus(percent);
}
```

- Uses parent class `bonus()` via `super`
- Overrides definition in parent class
- Consider the following assignment
`Employee e = new Manager(...);`
- Can we invoke `e.setSecretary()`?
 - `e` is declared to be an `Employee`
 - Static typechecking — `e` can only refer to methods in `Employee`
- What about `e.bonus(p)`? Which `bonus()` do we use?
 - Static → use `Employee.bonus()`
 - Dynamic → use `Manager.bonus()`
- **Dynamic dispatch** (dynamic binding, late method binding, ...) turns out to be more useful
 - Default in Java, optional in languages like C++ (`virtual` function)

Polymorphism

```
Employee[] emparray = new Employee[2];
Employee e = new Employee(...);
Manager m = new Manager(...);

emparray[0] = e;
emparray[1] = m;

for(int i = 0; i < emparray.length; ++i) {
    System.out.println(emparray[i].bonus(5.0));
}
```

- Every `Employee` in `emparray` “knows” how to calculate its `bonus` correctly
- Recall the event simulation loop that motivated Simula to introduce objects
- Also referred to as runtime polymorphism or inheritance polymorphism

Functions, signatures and overloading

- Signature of a function is its name and the list of argument types
- Can have different functions with the same name and different signatures
 - For example, multiple constructors
- Java class `Arrays` has a method `sort` to sort arbitrary scalar arrays
- Made possible by overloaded methods defined in class `Arrays`

```
double[] darr = new double[100];
int[] iarr = new int[500];
...
Arrays.sort(darr); // Sort the contents of darr
Arrays.sort(iarr); // Sort the contents of iarr

class Arrays {
    ...
    public static void sort(double[] a) { ... } // Sort arrays of double[]
    public static void sort(int[] a) { ... } // Sort arrays of int[]
    ...
}
```

- **Overloading:** multiple methods, same name, different signatures (different parameters), choice is static
- **Overriding:** multiple methods, same name, same signature, choice is static

- `Employee.bonus()`
- `Manager.bonus()`
- **Dynamic dispatch:** multiple methods, same signature, choice made at run-time

Type casting

- Consider the following assignment

```
Employee e = new Manager(...)
```

- Can we get `e.getSecretary()` to work?
 - Static type checking disallows this

- Type casting — convert e to Manager

```
((Manager) e).setSecretary(s)
```

- Cast fails (errors at run time) if `e` is not a `Manager`
- Can test if `e` is a `Manager`

```
if(e instanceof Manager) { ((Manager) e).setSecretary(s); }
```

- We can also use type casting for basic data types

```
double d = 29.98;
```

```
int nd = (int) d;
```

Summary

- A sub-class can override a method from a parent class
- Dynamic dispatch ensures that the most appropriate method is called, based on the run-time identity of the object
- Run-time/inheritance polymorphism, different from overloading
 - We will later see another type of polymorphism, *structural polymorphism*
 - For instance, use the same sorting functions for array of any datatype that supports a comparison operation
 - Java uses the term *generics*
- Use type-casting (and reflection) overcome static type restrictions